

AI Competitive Intelligence & Proposal Generator

Project Proposal for UCS503P

Thapar Institute of Engineering and Technology

Team members:

1. Aditya Kochar 1024030310
2. Kumar Ketubh 1024030311
3. Jaskaran Singh 1024031181

August 16, 2026

1 Higher-order goal

This project aims to improve strategic decision-making efficiency for students, startup founders, and agile product teams by turning scattered competitor changes into evidence-backed, actionable market intelligence. While the strategic domain is broad, the immediate engineering objective is to translate multi-source web change detection, agentic context verification, and automated proposal generation into a measurable, deployable software platform with zero paid LLM dependencies.

2 Time-to-value

We will deliver meaningful increments quickly by prototyping the core change-detection, verification, and proposal generation workflows, validating them with users within a short iteration window.

We will prioritize fast feedback over perfection by using weekly iterations (and targeting CI-backed automation early) to reduce release friction.

- Success is defined by what can be delivered and measured in the first iteration, then improved based on user feedback.

3 Problem Statement

Developers, product leads, and student teams face significant friction and manual overhead when tracking competitor moves across fragmented web channels (e.g., pricing tables, product feature pages, changelogs, blogs, and public repositories). Common pain points include:

- **Fragmentation:** Competitor data is dispersed across multiple URLs and unstandardized formats, causing lost context and high manual research overhead.
- **Low transparency:** Existing website-monitoring tools notify teams that a DOM change occurred, but fail to explain why the change matters, what evidence supports it, or what strategic action should be considered.
- **Slow coordination:** Teams lack an automated, end-to-end mechanism to convert raw market observations directly into structured, actionable project proposals.
- **Poor data quality & hallucinations:** Raw web scrapers capture thousands of irrelevant nodes (styling, navigation, ads) that introduce noise, inflate token costs, and trigger AI hallucinations when unsubstantiated by raw factual diffs.

Why this matters now (contextual relevance): In fast-moving technology environments with limited developer bandwidth, an agentic intelligence system that autonomously filters trivial diffs, validates factual evidence, and drafts structured proposals substantially reduces research latency and repetitive manual documentation.

4 Proposed Solution

4.1 Overview

Software Proposition

Build a web-based "AI Competitive Intelligence & Proposal Generator" with the following components:

- **Workspace & Competitor Source Portal:** users define their product context, onboard 2–5 competitors, and configure public URLs (pricing, product, changelogs, blogs, RSS, GitHub).
- **Deterministic Change Ingestion & Triage:** automated noise filtering, snapshot content hashing, and classification of changes into trivial, relevant, or high-impact items.
- **Agentic Verification & Impact Analysis:** multi-step LangGraph workflow with separate Context Research, Impact Analysis, and Critic/Verifier nodes to challenge unsupported claims against raw source snippets.
- **Structured Proposal Generator:** synthesize modular, evidence-grounded project proposals from verified intelligence items without paid API dependencies.
- **Admin/Ops & Intelligence Dashboard:** central review interface with confidence indicators (Low/Medium/High), evidence timelines, scan run histories, and proposal editors.

4.2 Core workflow

1. User signs in, creates a workspace with product positioning, and configures target competitor sources.
2. User triggers an on-demand scan; collectors fetch web content and compute diffs against stored snapshots.
3. Deterministic filters remove trivial DOM changes; candidate items pass to the LangGraph state machine.
4. Agent pipeline executes: Context Researcher gathers corroborating evidence → Impact Agent evaluates workspace relevance → Critic Node verifies factual support (Verified → Output Finding; Unsubstantiated → Downgrade/Reject).
5. User reviews validated findings on the dashboard, selects key intelligence items, and generates an editable, structured proposal.

4.3 Operational constraints for an educational setting

- Keep the solution usable on standard development machines via lightweight backend execution.
- Avoid dependence on advanced research algorithms or paid proprietary LLM APIs; enforce zero-cost/open-source model adapters.
- Design for deployability using common web hosting, managed databases (PostgreSQL), and modular service architectures.

5 Solution Approach

Engineering focus

This is an engineering course project, so we focus on scalable administrative and workflow aspects rather than novel algorithm research.

5.1 DOM pruning / quality assistance...non-research

DOM and web content filtering will be heuristic-based:

- Normalize structured elements (pricing blocks, release headers, changelog lists, article bodies).
- Use simple deterministic hashing and readability-style text extractors (BeautifulSoup/feedparser) without claiming state-of-the-art scraping complexity.
- Supply only candidate diffs and extracted text excerpts to the agentic verification pipeline; never feed raw, unparsed HTML blobs to LLMs.

5.2 Web architecture...web-based solution

A typical 3-tier web architecture:

- **Frontend:** responsive Next.js web application (workspace setup, source configuration, findings review, proposal generator/editor).
- **Backend API & Agent Runtime:** FastAPI service handling REST endpoints, auth, and stateful LangGraph orchestration pipelines.
- **Database / Storage:** PostgreSQL storing workspaces, competitors, sources, historical snapshots, scan runs, evidence snippets, and generated proposals.

5.3 CI/CD and fast delivery enabler

CI/CD will be used to:

- Automatically build and run unit tests on every change (source parsers, hash diffing, schema validation).
- Produce repeatable deployment artifacts to staging (e.g., using Docker (optional)).
- Enable safe and frequent releases of incremental features.

6 Evaluation Criterion... measurable and attributable

We define success using measurable metrics tied to the core workflow.

6.1 Primary evaluation metric

Evidence Grounding Accuracy (EGA): percentage of generated proposal claims and intelligence findings directly substantiated by retained source snapshot evidence without hallucination.

- **Target:** median \$EGA > 85\%\$ across benchmark test scenarios during the pilot evaluation window.
- **Attribution:** measured directly by automated quote-matching against snapshot text and validated by human evaluators.

6.2 Secondary evaluation metrics

- **Research time reduction:** reduce competitor analysis and proposal drafting time compared to baseline manual workflows (collected via survey and timing logs).
- **Change detection precision/recall:** accuracy in identifying genuine pricing/feature updates against controlled website test sets.
- **User clarity:** user-reported clarity of impact assessments and evidence timelines using a simple 1–5 feedback score.
- **Reliability:** availability of the scan engine and web API during testing windows (e.g., $\geq 99\%$ uptime in pilot).

6.3 Pilot validation plan... high-level

- Recruit 10–20 pilot users (students/founders) and 3–5 competitor benchmark domains (tracking simulated pricing and changelog releases).
- Compare metrics before/after within the iteration window by logging baseline research time estimates via short interviews and recorded observation.

7 Scalability... with foresight

The proposition should scale in both theoretical and practical senses.

1. **Scalable (theoretically):** The system should support growth in the number of monitored sources and concurrent scan runs by:

- decoupling components (Next.js frontend / FastAPI backend / LangGraph worker / PostgreSQL),
- enabling pagination and indexing on snapshot hashes and workspace IDs for common queries,
- using stateless REST API design with modular provider interfaces.

2. **Operationally deployable (practically):** The system should run on common web hosting:

- managed PostgreSQL database,
- containerized (e.g., Docker (optional)) or platform-hosted deployment,
- CI/CD pipeline for staging/production.

8 Engine availability heuristic

A mature set of "engines" (tooling/services) is available to implement this as a web-based project:

- Web framework for REST APIs and responsive UI (FastAPI and Next.js).
- Agent orchestration and workflow engine (LangGraph / LangChain ecosystem).
- Web content extraction utilities (BeautifulSoup, feedparser, standard HTTP clients).
- Managed relational database (PostgreSQL with SQLAlchemy/Prisma ORM).
- Authentication approach (JWT token-based or secure session-based auth).
- Container runtime (Docker (optional)) and CI pipeline runners.
- Test framework for unit/integration tests (pytest / jest).

We will use these mature components to focus on workflow design, correctness, and measurement rather than inventing new infrastructure.

9 Project scope and deliverables

9.1 Initial deliverable... first iteration

iteration-friendly

- Workspace setup & competitor source configuration forms.
- Worker workflow: HTTP fetcher, snapshot content hasher, and deterministic diff detector.
- Agent pipeline: LangGraph research-verify chain using free/open-source LLM adapters.
- Basic scan history dashboard showing validated findings and evidence timeline.
- Markdown proposal generation from selected findings.
- CI: build + backend unit tests + linting.
- CD to staging with a single command or pipeline job (with Docker (optional)).

9.2 Subsequent deliverables

- In-browser interactive proposal editor with section-by-section regeneration.
- Advanced anomaly filtering for dynamic JS-heavy websites.
- Notification layer (email/webhook alerts on high-severity competitor updates).
- Admin dashboard for aggregate metrics, grounding accuracy scores, and scan backlogs.
- Improved search/filtering and indexed database queries for scale readiness.

10 Risks and mitigations

- **LLM Hallucinations / Unsupported Claims:** isolate AI reasoning behind a strict Critic/Verifier node; mandate that every market claim links to an exact text excerpt and snapshot URL.
- **Rate Limits on Free LLM Providers:** implement deterministic candidate filtering before AI calls, caching by snapshot hash, bounded exponential backoffs, and fallback heuristic reporting.
- **User Adoption & Usability:** provide an intuitive Next.js interface with pre-built source templates and straightforward proposal export options.
- **Evaluation Measurement Ambiguity:** establish automated benchmark datasets with pre-labeled diffs and evidence-grounding rubrics before testing begins.
- **Operational Issues in Pilot:** use staging early; ensure CI/CD produces repeatable deployments with environment-variable-driven model switching.

11 Summary

This project proposes a deployable web platform to reduce the manual effort and fragility of competitor market monitoring and proposal drafting. It meets the course criteria by emphasizing time-to-value through rapid iterations, implementing CI/CD to support frequent safe releases, and using measurable evaluation criteria (especially Evidence Grounding Accuracy). It remains focused on engineering workflow and scalability readiness rather than research-driven novelty.